

CÂU 1:

```
1 void add_edge(Graph* G, int x, int y) {
2     if (G->m < MAX_EDGES) {
3         G->edges[G->m].x = x;
4         G->edges[G->m].y = y;
5         G->m++;
6     } else {
7         printf("Danh sách đầy\n");
8     }
9 }
```

CÂU 2:

Answer: (penalty regime: 0, 25, ... %)

```
1 int degree(Graph* G, int x) {
2     int count = 0;
3
4     for (int i = 0; i < G->m; ++i) {
5         if (G->edges[i].x == x || G->edges[i].y == x) {
6             count++;
7         }
8     }
9
10    return count;
11 }
```

CÂU 3:

Answer: (penalty regime: 0, 25, ... %)

```
1 void add_edge(Graph* G, int x, int y) {
2     if (x < 1 || x > G->n || y < 0 || y > G->n)
3         return;
4     G->edges[G->m].x = x;
5     G->edges[G->m].y = y;
6     G->m++;
7 }
```

CÂU 4:

Answer: (penalty regime: 0, 25, ... %)

```
1 void add_edge(Graph* G, int e, int x, int y) {
2     G->A[x][e] = 1;
3     G->A[y][e] = 1;
4     G->m++;
5 }
```

CÂU 5:

Answer: (penalty regime: 0, 25, ... %)

```
1 int deg(Graph* G, int x) {
2     int e, deg = 0;
3     for (e = 1; e <= G->m; e++)
4         if (G->A[x][e] != 0)
5             deg++;
6     return deg;
7 }
```

CÂU 6:

Answer: (penalty regime: 0, 25, ... %)

```
1 void init_graph(Graph* G, int n, int m){
2     int i,j;
3     G->n=n;
4     G->m=m;
5     for(i=1;i<=G->n;i++)
6         for(j=1;j<=G->n;j++)
7             G->A[i][j]=0;
8 }
```

CÂU 7:

Answer: (penalty regime: 0, 25, ... %)

```
1 void add_edge(Graph* G, int x, int y) {
2     G->A[x][y]++;
3     G->A[y][x]++;
4     G->m++;
5 }
```

CÂU 8:

Answer: (penalty regime: 0, 25, ... %)

```
1 void add_edge(Graph* G, int x, int y) {
2     G->A[x][y]=1;
3     G->m++;
4 }
```

CÂU 9:

Answer: (penalty regime: 0, 25, ... %)

```
1 void add_edge(Graph* G, int x, int y) {
2     G->A[x][y]=1;
3     G->A[y][x]=1;
4     G->m++;
5 }
```

CÂU 10:

Answer: (penalty regime: 0, 25, ... %)

```
1 int deg(Graph* G, int x) {
2     int y, deg=0;
3     for(y=1; y<=G->n; y++)\
4         if(G->A[x][y]!=0)
5             deg++;
6     return deg;
7 }
```

CÂU 11:

Answer: (penalty regime: 0, 25, ... %)

```
1 int edgeCount(Graph* G) {
2     int count = 0;
3     for(int i=1; i<=G->n; i++)
4         for(int j=1; j<=G->n; j++) {
5             count+=G->A[i][j];
6         }
7     return count / 2;
8 }
```

CÂU 12

Answer: (penalty regime: 0, 25, ... %)

```
1 List neighbors(Graph* G, int x){
2     List L;
3     int y;
4     for(y=1; y<=G->n; y++)
5         if(G->A[x][y]!=0)
6             push_back(&L, y);
7     return L;
8 }
```

CÂU 13:

Answer: (penalty regime: 0, 25, ... %)

```
1 void remove_edge(Graph* G, int x, int y) {
2     if(G->A[x][y]==0)
3         return;
4     G->A[x][y]-=1;
5     G->A[y][x]-=1;
6     G->m--;
7 }
```

CÂU 14:

Answer: (penalty regime: 0, 25, ... %)

```
1 void add_edge(Graph* G, int e, int x, int y) {
2     G->A[x][e]+=1;
3     G->A[y][e]+=1;
4     G->m++;
5 }
```

CÂU 15:

Answer: (penalty regime: 0, 25, ... %)

```
1 void add_edge(Graph* G, int x, int y) {
2     G->A[x][y]+=1;
3     G->A[y][x]+=1;
4     G->m++;
5 }
```